# UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

## FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

## ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS




*LAB 04 - Arquitectura Zero Trust*

**CURSO:**

TOPICOS AVANZADOS EN INGENIERIA DE SOFTWARE

**DOCENTE:**

ROBERT EDISON ARISACA MAMANI

**PRESENTADO POR:**

JEAN CARLO CHARA CONDORI

VLADIMIR JAWARD HANCCO SONCCO

GONZALO RAIL LAYME MAMANI

JEANPIER MICHAELSON VALERA YANA

**AREQUIPA – PERÚ**

**2026**

# Índice de Contenidos

1. Alcance del Proyecto y Enlaces de Evaluación . . . 3
Cuadro de Referencia Operativa para Evaluación Docente . . . 3
Delimitación del Alcance y Limitaciones Free Tier . . . 3
2. Arquitectura de Seguridad Zero Trust (NIST SP 800-207) . . . 3
Principios Fundamentales (NIST SP 800-207) . . . 3
Arquitectura Lógica de Control y Datos (PA, PEP, PDP) . . . 4
3. Infraestructura Cloud y Gobernanza de Costos . . . 4
Control Preventivo de Presupuesto (AWS Budgets) . . . 4
Aprovisionamiento de Usuarios IAM por Integrante (CA-NF03.3) . . . 5
Recursos Físicos Desplegados en AWS `us-east-2` . . . 5
4. Estructura del Monorepo y Git Flow . . . 6
Estructura Modular del Monorepo . . . 7
Integración Continua (GitHub Actions) . . . 7
5. Implementación de Controles Críticos . . . 8
Autorización en el Backend: Policy Decision Point (PDP) . . . 8
Idempotencia Atómica con TransactWriteItems (RNF-HU-04) . . . 8
Seguridad en el Cliente SPA: Tokens en Memoria y Storage Purge Guard (RNF-HU-11) . . . 9
Ciclo de Vida y Limitación de Revocación de Tokens JWT (CA-03.3, CA-03.4) . . . 9
Interfaces de Usuario de la Plataforma . . . 10
Medición Experimental de Rendimiento y Accesibilidad . . . 10
6. Verificación Experimental y Suite de Pruebas (T01 a T14) . . . 11
Consolidado Global de Métricas de Verificación . . . 11
Matriz de Casos de Prueba de Aceptación (T01 a T14) . . . 11
7. Observabilidad, Auditoría y Cumplimiento Normativo . . . 17
Observabilidad sin Fuga de Secretos (RNF-HU-07) . . . 17
Declaración de Asistencia de Inteligencia Artificial (AI_USAGE.md) . . . 17
8. Conclusiones y Plan de Desmantelamiento . . . 17
Conclusiones Técnicas del Proyecto . . . 17
Plan de Limpieza y Desmantelamiento (RNF-HU-09) . . . 18
Referencias Bibliográficas . . . 18
Glosario de Términos y Tecnologías Clave . . . 18

# 1. Alcance del Proyecto y Enlaces de Evaluación

El presente informe documenta el diseño, despliegue y verificación experimental de la plataforma cloud-native serverless «Fundación AQP-SOLIDARIA», orientada a la publicación de campañas sociales y recepción de intenciones de donación con trazabilidad estricta.

La plataforma se fundamenta en el modelo de seguridad **Zero Trust Architecture (ZTA)** estipulado en **NIST SP 800-207** [1] y **NIST SP 800-207A** [2], asumiendo que la red perimetral está comprometida («assume breach»), requiriendo verificación continua de identidad y permisos por recurso, y aplicando privilegio mínimo en usuarios y cargas de trabajo.

### Cuadro de Referencia Operativa para Evaluación Docente

Para facilitar la verificación independiente y reproducción en vivo de la plataforma:

| Recurso de Evaluación | Identificador / URL Pública | Credenciales / Notas |
|---|---|---|
| **Repositorio GitHub** | https://github.com/vhanccos/secure-donations-2026 | Monorepo con CI Actions en verde |
| **API Gateway PEP (Live)** | https://evft7i355m.execute-api.us-east-2.amazonaws.com/academic | Stage `academic` (TLS 1.3 / HSTS) |
| **Cognito Hosted UI (SSO)** | https://aqp-solidaria-915275802786.auth.us-east-2.amazoncognito.com | OAuth 2.0 PKCE + TOTP MFA |
| **Usuario Donante (Prueba)** | `challenger_donante_a@aqp-solidaria.org` | `TestPassword123!` (Sin MFA) |
| **Usuario Gestor (MFA)** | `challenger_gestor@aqp-solidaria.org` | `TestPassword123!` + TOTP RFC 6238 |
| **Usuario Auditor (MFA)** | `challenger_auditor@aqp-solidaria.org` | `TestPassword123!` + TOTP RFC 6238 |
| **Ejecución de Pruebas E2E** | `bash tests/run-all-tests.sh` | 55/55 pruebas en vivo aprobadas |
| **Prueba Individual** | `node tests/run-single-test.js T02` | Soporta escenarios T01 a T14 |

### Delimitación del Alcance y Limitaciones Free Tier

Conforme a la especificación académica de `requirements.md` (§1):

- **Donaciones Estrictamente Simuladas:** Prohibición absoluta de solicitar, transmitir o almacenar números de tarjeta, CVV o cuentas bancarias. Las intenciones se registran como `PENDIENTE_SIMULACION` y se confirman únicamente mediante un endpoint transaccional exclusivo del stage académico.
- **Limitaciones Declaradas por Restricciones de AWS Free Plan:**
  1. **Postura del Dispositivo Cliente:** No se dispone de agentes EDR corporativos en los hosts que abren la SPA.
  2. **Adaptive Risk Authentication:** La cuenta Free Tier no incluye Cognito Advanced Security para scoring heurístico ML por sesión.
  3. **ZTNA Gestionado Dedicado:** El control perimetral se resuelve en API Gateway HTTP API v2 con JWT Authorizer, en lugar de AWS Verified Access.
  4. **SIEM Gestionado:** La detección se apoya en CloudWatch Logs estructurados y CloudTrail inmutable, sin GuardDuty ni Security Hub.

# 2. Arquitectura de Seguridad Zero Trust (NIST SP 800-207)

### Principios Fundamentales (NIST SP 800-207)

A diferencia de los modelos perimetrales tradicionales («castillo y foso»), la arquitectura implementada aplica de forma verificable los 7 principios de la publicación especial NIST SP 800-207 [1]:

| Principio NIST SP 800-207 | Implementación Técnica en la Plataforma | Evidencia / Verificación |
|---|---|---|
| 1. Todo dato y servicio es un recurso | Cada endpoint HTTP de API Gateway y cada tabla de DynamoDB cuenta con políticas de acceso explícitas y roles IAM independientes. | Políticas IAM mínimas por función SAM; pruebas T01, T02, T10 |
| 2. Comunicación segura independiente de la red | Cifrado TLS 1.3 / HTTPS obligatorio de extremo a extremo. Cabeceras HSTS, CSP estricto y denegación de tráfico plano HTTP. | CA-NF01.1; T01.3 (HSTS, nosniff, DENY) |

| Principio NIST SP 800-207 | Implementación Técnica en la Plataforma | Evidencia / Verificación |
|---|---|---|
| 3. Acceso por sesión y recurso individual | Access tokens JWT con vigencia $\leq 15$ min, scopes OAuth 2.0 por ruta y validación de propiedad por recurso (`sub`). | T09 (BOLA/IDOR mitigado), T11 (Scopes), T13 (Expiración) |
| 4. Política dinámica y contextual | Evaluación en tiempo de ejecución combinando identidad autenticada, grupo de Cognito, estado del proyecto y stage de despliegue. | T03 (403 GESTOR requerido), T08, T12 (Segregación) |
| 5. Monitoreo de activos e integridad | Limitación declarada del Free Tier (sin inspección ZTNA de endpoint físico); mitigado con CloudTrail y auditoría estricta en CI. | Sección 3 / 7; logs estructurados y CloudTrail |
| 6. Autenticación y autorización estrictas | JWT Authorizer en PEP + validación RBAC/Ownership en Lambda PDP + MFA TOTP obligatorio para `GESTOR` y `AUDITOR`. | T02 (401 PEP), T11 (Firma), T14 (MFA TOTP obligatorio) |
| 7. Recolección de telemetría continua | Logs estructurados JSON en CloudWatch sin secretos, tabla dedicada `AuditEvents` en DynamoDB y trazabilidad por `requestId`. | CA-NF07.1, CA-NF07.2; T12.4 (Auditoría) |

### Arquitectura Lógica de Control y Datos (PA, PEP, PDP)

Siguiendo los estándares de NIST SP 800-207 y CISA ZTMM v2.0 [4], el sistema desacopla con rigor el plano de control del plano de datos:

1. **Policy Administrator (PA):** Amazon Cognito User Pool (`us-east-2_jSTLexY2N`) administra identidades federadas, autenticación multifactor (TOTP), y emite tokens JWT de vida corta (15 min) mediante flujos seguros PKCE.
2. **Policy Enforcement Point (PEP):** Amazon API Gateway HTTP API v2 intercepta toda solicitud entrante como punto de control perimetral, validando la firma RSA, el issuer y los scopes OAuth requeridos (`donations:write`, etc.) antes de invocar cualquier cómputo.
3. **Policy Decision Point (PDP):** Funciones AWS Lambda (Node.js 24 + TypeScript) ejecutan el módulo de autorización (`authz/pdp.ts`), evaluando los claims criptográficos (`sub`, `cognito:groups`) y las restricciones de negocio para prevenir vulnerabilidades de autorización a nivel de objeto (BOLA / OWASP API1:2023 [5]).

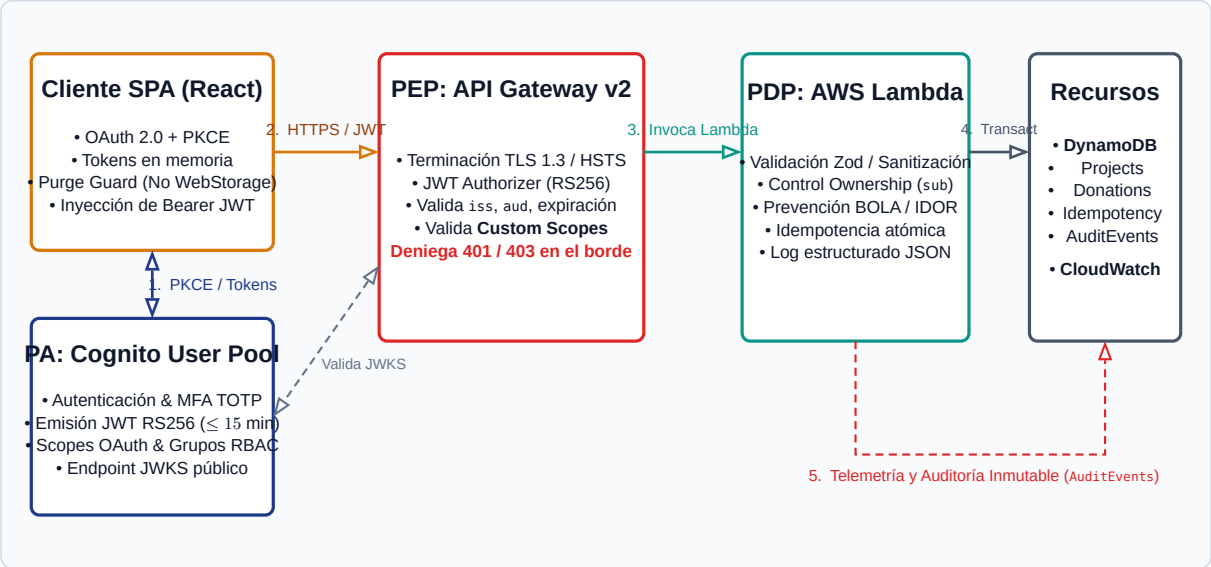



Figura 1: Flujo lógico de transacciones y evaluación de políticas bajo el modelo Zero Trust (NIST SP 800-207: PA, PEP, PDP y PIP).

## 3. Infraestructura Cloud y Gobernanza de Costos

En cumplimiento estricto con el requerimiento **RNF-HU-09 (CA-NF09.1)**, la infraestructura opera bajo el plan gratuito de AWS (Free Tier), asegurando costo cero y estricto control de privilegios de acceso.

### Control Preventivo de Presupuesto (AWS Budgets)

1. **Protección de Cuenta Root:** Se activó autenticación multifactor (MFA TOTP) en la cuenta administradora y se revocaron permanentemente todas las llaves de acceso estáticas (access keys).

2. **Alarma Preventiva en AWS Budgets:** Se configuró un presupuesto mensual con umbral de **$1.00 USD** en la región `us-east-2`, con notificaciones automatizadas vía correo electrónico al 50%, 80% y 100% del consumo proyectado.

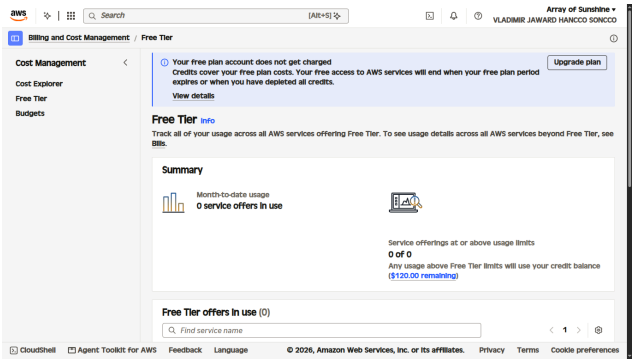



Figura 2: Alarma de costo cero configurada en AWS Budgets con umbral de $1.00 USD.

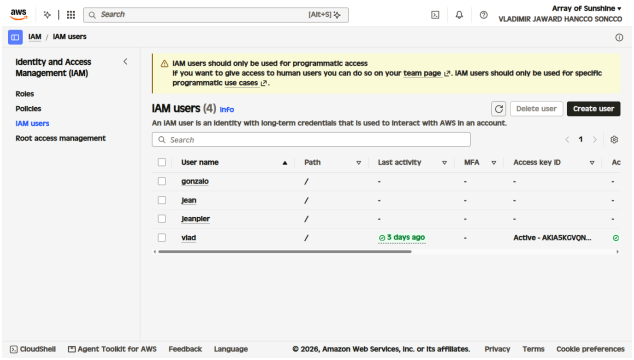



Figura 3: Usuarios IAM individuales aprovisionados para los 4 integrantes del equipo.

## Aprovisionamiento de Usuarios IAM por Integrante (CA-NF03.3)

Para garantizar la trazabilidad individual y erradicar el uso compartido de credenciales, se aprovisionaron cuatro usuarios en AWS IAM con permisos acotados:

| Integrante | Usuario IAM | Identificador / Correo | Rol de Ingeniería Asignado |
|---|---|---|---|
| Vladimir Hancco | `vlad` | vhanccos (`vhanccos@gmail.com`) | Líder Técnico, Cloud Infrastructure & DevOps (SAM, CI/CD, IAM) |
| Jean Chara | `jean` | `jcharaco@unsa.edu.pe` | Ingeniero Backend Core (Idempotencia atómica, DynamoDB, Transacciones) |
| Jeanpier Valera | `jeanpier` | `jvalera@unsa.edu.pe` | Arquitecto de Seguridad e Identidad (Cognito, MFA, Threat Model) |
| Gonzalo Layme | `gonzalo` | `glaymem@unsa.edu.pe` | Frontend Lead & Accesibilidad (React 19, OIDC tokens en memoria) |

## Recursos Físicos Desplegados en AWS `us-east-2`

La infraestructura en la nube está completamente desplegada y operativa, interactuando en vivo con la suite de pruebas automatizadas:

| Componente Arquitectural | Servicio AWS | Identificador Físico / Configuración en us-east-2 |
|---|---|---|
| PEP (API Gateway) | Amazon API Gateway (HTTP v2) | `https://evft7i355m.execute-api.us-east-2.amazonaws.com/academic` |
| PA (Identity Provider) | Amazon Cognito User Pool | `us-east-2_jSTLexY2N` (Grupos: DONANTE, GESTOR, AUDITOR) |
| App Client (SPA PKCE) | Cognito User Pool Client | `6qc17dq9iqpoqq8e4sk5doqpnn` (AccessTokenValidity: 15 min) |
| Auth Domain Hosted UI | Amazon Cognito Domain | `https://aqp-solidaria-915275802786.auth.us-east-2.amazoncognito.com` |
| Catálogo de Proyectos | Amazon DynamoDB | Tabla `Projects` (PK: `projectId` [S], GSI: `status-index` [S]) |
| Registro de Donaciones | Amazon DynamoDB | Tabla `Donations` (PK: `donationId` [S], GSI: `donorSub-index` [S]) |
| Idempotencia Transaccional | Amazon DynamoDB | Tabla `Idempotency` (PK: `pk` [S], TTL activo en `expiresAt` [N]) |
| Auditoría Inmutable | Amazon DynamoDB | Tabla `AuditEvents` (PK: `eventType` [S], SK: `timestamp#id` [S]) |
| PDP (Cómputo Backend) | AWS Lambda (Node.js 24) | Roles IAM de menor privilegio (`ListProjectsFunctionRole`, etc.) |

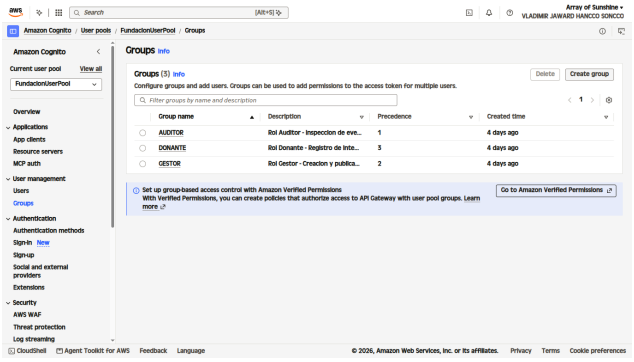



Figura 4: Amazon Cognito User Pool us-east-2_jSTLexY2N y grupos de seguridad.

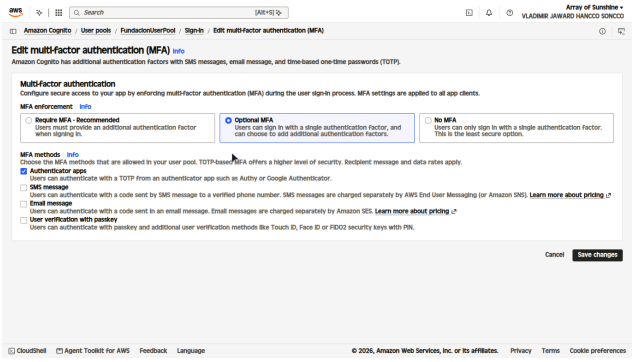



Figura 5: Configuración de TOTP MFA obligatorio para roles administrativos.

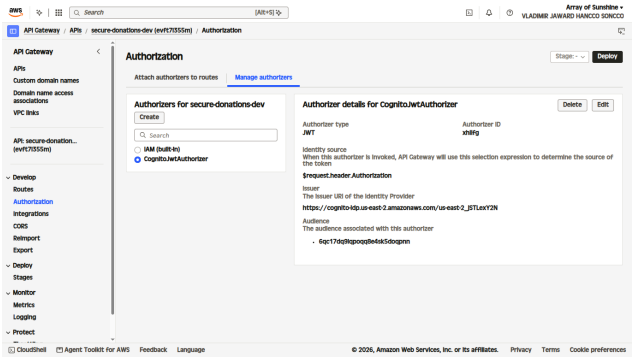



Figura 6: API Gateway HTTP API v2 con JWT Authorizer configurado como PEP.

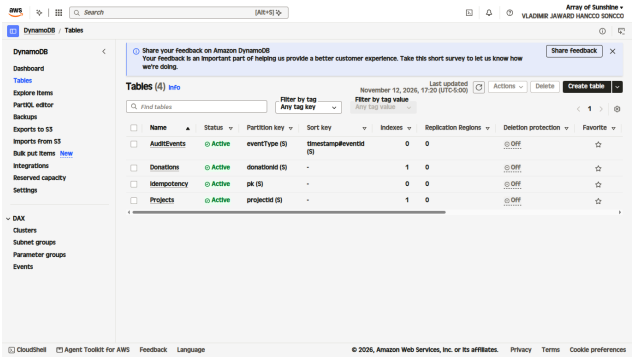



Figura 7: Tablas DynamoDB en modo On-Demand (Projects, Donations, Idempotency, Audit).

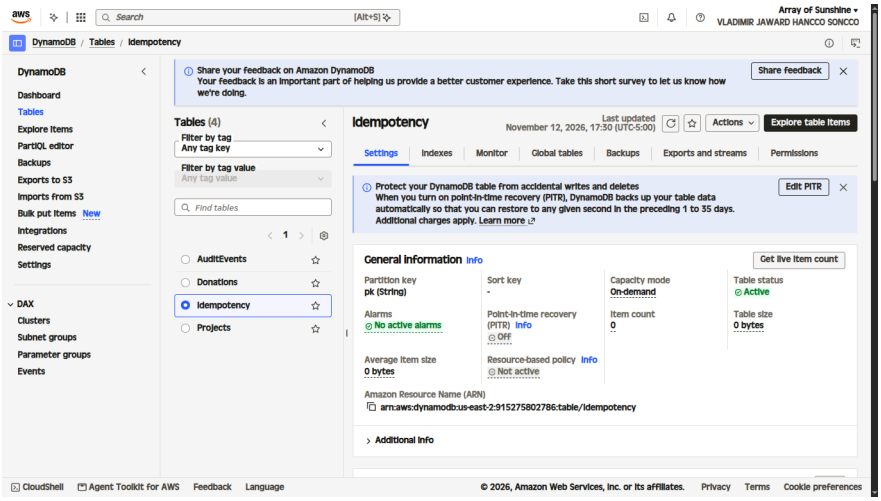



Figura 8: Configuración de Time to Live (TTL) activo en el atributo expiresAt de la tabla Idempotency.

## 4. Estructura del Monorepo y Git Flow

El proyecto se estructuró bajo un monorepo modular versionado en GitHub (`secure-donations-2026`). Cada integrante asumió un rol de especialización y desarrolló sus funcionalidades en ramas de características independientes con autoría verificable en el registro de commits:

**Vladimir Hancco** — Arquitectura Cloud & DevOps

- Infraestructura como Código Serverless (AWS SAM CLI)
- Configuración Cognito User Pool, Grupos y MFA TOTP
- HTTP API Gateway v2, JWT Authorizer & Scopes PEP
- Seguridad AWS Free Tier ($1.00 USD Budget) y CI/CD

**Jean Chara** — Backend & Lógica de Negocio

- Handlers AWS Lambda en Node.js 24 + TypeScript
- Transacciones atómicas e idempotencia (`TransactWrite`)
- Esquemas de validación Zod y mitigación de inyección
- Telemetría JSON estructurada y trazas en CloudWatch

**Jeanpier Valera** — Frontend SPA & Seguridad Cliente

- SPA interactiva en React 19 + TypeScript + Tailwind CSS
- Almacenamiento volátil de tokens en memoria (React Context)
- Purga preventiva de WebStorage contra ataques XSS
- Flujo PKCE OAuth 2.0 y modal interactivo para MFA TOTP

**Gonzalo Layme** — QA, Pruebas Negativas & Auditoría

- Batería automatizada de pruebas T01 a T14 (55 tests)
- Ensayos negativos: BOLA/IDOR, manipulación de tokens y scopes
- Verificación de eventos inmutables en tabla `AuditEvents`
- Validación de matrices de cumplimiento NIST SP 800-207

Figura 9: Distribución de roles, módulos técnicos y responsabilidades entre los 4 integrantes del equipo.

## Estructura Modular del Monorepo

Para garantizar separación de responsabilidades y trazabilidad completa:

| Módulo / Directorio | Tecnología Principal | Responsabilidad y Contenido |
|---|---|---|
| `template.yaml` | AWS SAM / CloudFormation | Definición declarativa de toda la infraestructura (Cognito, API Gateway, DynamoDB, Lambdas e IAM). |
| `backend/src/` | Node.js 24 + TypeScript | 9 funciones Lambda, PDP (`auth/pdp.ts`), validaciones Zod e idempotencia DynamoDB. |
| `frontend/src/` | React 19 + Tailwind CSS + Vite | Vistas SPA, cliente OIDC con PKCE, `Storage Purge Guard` y tokens volátiles en memoria. |
| `tests/` | Node.js Test Runner + Vitest | Suite E2E automatizada (T01 a T14), pruebas de concurrencia masiva y runner individual. |
| `informe/` | Typst 0.15.1 + CeTZ | Documentación técnica reproducible, diagramas vectoriales nativos y evidencias fotográficas. |

## Integración Continua (GitHub Actions)

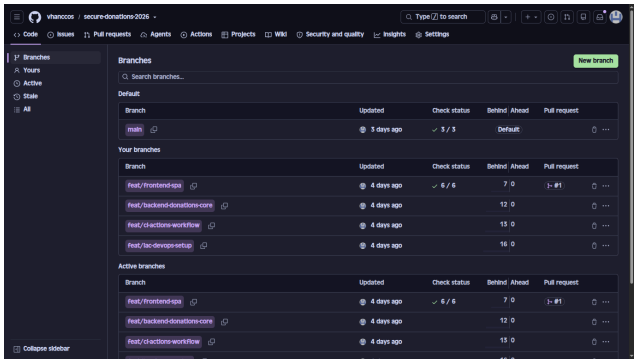



Figura 10: Historial de ramas colaborativas y Pull Requests por integrante en GitHub.

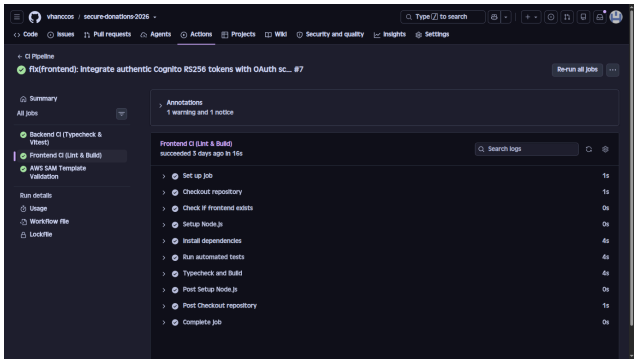



Figura 11: Pipeline de CI en GitHub Actions con validación SAM, linting y pruebas.

El flujo de integración continua (`.github/workflows/ci.yml`) evalúa automáticamente en cada Pull Request:

1. Validación estructural de la plantilla Serverless con **AWS SAM CLI (`sam validate`)**.
2. Comprobación estricta de tipos estáticos con **TypeScript Compiler (`tsc --noEmit`)**.
3. Análisis estático y reglas de estilo de código con **ESLint 10**.
4. Batería de pruebas unitarias automatizadas con **Vitest v3.2**.
5. Auditoría de seguridad de dependencias npm (`npm audit --audit-level=high`).

## 5. Implementación de Controles Críticos

### Autorización en el Backend: Policy Decision Point (PDP)

Bajo Zero Trust, la aplicación desconfía por completo de cualquier parámetro de identidad enviado en el cuerpo de la solicitud (prevención de OWASP API1:2023 - BOLA). En el backend ([`backend/src/auth/pdp.ts`](file:

TypeScript — Módulo PDP: Extracción de Claims Criptográficos y Control de Propiedad — TypeScript

```typescript
export function extractAuthContext(event: APIGatewayProxyEventV2): AuthContext {
  const claims = (event.requestContext as any)?.authorizer?.jwt?.claims;
  if (!claims || !claims.sub) {
    throw new HttpError(401, 'UNAUTHORIZED', 'Missing or invalid authentication token');
  }
  return {
    sub: claims.sub,
    groups: Array.isArray(claims['cognito:groups']) ? claims['cognito:groups'] : [],
  };
}

export function assertOwnership(callerSub: string, resourceOwnerSub: string): void {
  if (callerSub !== resourceOwnerSub) {
    throw new HttpError(403, 'FORBIDDEN', 'Acceso denegado: no es propietario de este recurso');
  }
}
```

### Idempotencia Atómica con TransactWriteItems (RNF-HU-04)

Para prevenir el doble aporte financiero o condiciones de carrera ante desconexiones de red, la creación de donaciones ejecuta una transacción atómica coordinada en DynamoDB:

TypeScript — Reserva de Idempotencia y Donación Atómica en DynamoDB — TypeScript

```typescript
await ddb.send(new TransactWriteCommand({
  TransactItems: [
    {
      Put: {
        TableName: process.env.IDEMPOTENCY_TABLE_NAME,
        Item: {
          pk: `${donorSub}#createDonation#${idempotencyKey}`,
          payloadHash,
          donationId,
          expiresAt: ttl24h
        },
        ConditionExpression: "attribute_not_exists(pk)"
      }
    },
    {
      Put: {
        TableName: process.env.DONATIONS_TABLE_NAME,
```

```
            Item: {
              donationId,
              projectId,
              donorSub,
              amountCents,
              currency: "PEN",
              status: "PENDIENTE_SIMULACION",
              createdAt: now
            }
          }
        }
      ]
    }));
```

Si la clave UUID ya existe con el mismo hash SHA-256 del payload, se retorna la respuesta previa en caché (`HTTP 201`). Si el payload difiere, se rechaza inmediatamente con `HTTP 409 Conflict`.

### Seguridad en el Cliente SPA: Tokens en Memoria y Storage Purge Guard (RNF-HU-11)

La aplicación SPA (React 19 + Tailwind CSS) implementa el flujo estándar **OAuth 2.0 Authorization Code con PKCE**. Para neutralizar por diseño los vectores de ataque de Cross-Site Scripting (XSS) y robo de credenciales:

- Los tokens JWT (Access y Refresh) se almacenan **exclusivamente en memoria volátil** de JavaScript (React Context).
- Un componente centinela (**Storage Purge Guard**) purga periódicamente `localStorage` y `sessionStorage`, impidiendo la persistencia en disco del navegador:

TypeScript — Centinela Storage Purge Guard contra ataques XSS

```typescript
const sensitiveKeys = ['accessToken', 'idToken', 'refreshToken', 'jwt', 'auth'];
for (const k of sensitiveKeys) {
  if (localStorage.getItem(k) !== null) {
    console.warn(`[Zero Trust Guard] Purged token from localStorage: ${k}`);
    localStorage.removeItem(k);
  }
}
```

### Ciclo de Vida y Limitación de Revocación de Tokens JWT (CA-03.3, CA-03.4)

Siguiendo los lineamientos de diseño de **Amazon API Gateway HTTP API v2** y **Amazon Cognito**:

- **Validación Local en el PEP:** El JWT Authorizer de API Gateway valida criptográficamente la firma RS256 contra el JWKS público, el emisor (`iss`), la audiencia/client ID y la fecha de expiración (`exp`) **de forma local sin invocar a Cognito por cada petición** para garantizar baja latencia ($< 15$ ms).
- **Limitación Demostrada de Revocación:** Como API Gateway no consulta activamente la lista de revocaciones en tiempo real, un **Access Token** revocado mediante el endpoint `/oauth2/revoke` continúa siendo válido en el PEP hasta su fecha programada de expiración.
- **Mitigación Zero Trust:** Para mitigar esta ventana de exposición y satisfacer el principio 3 de NIST SP 800-207, la vigencia del **Access Token** se configuró estrictamente en un máximo de **15 minutos** (`AccessTokenValidity: 15`). Adicionalmente, el **Refresh Token** sí es revocado de forma inmediata en Cognito, impidiendo que el cliente renueve credenciales una vez cerrada la sesión. Esta limitación y su mitigación quedan empíricamente verificadas en la suite de pruebas mediante el escenario **T13** (`T13.1`, `T13.2`, `T13.3`).

## Interfaces de Usuario de la Plataforma

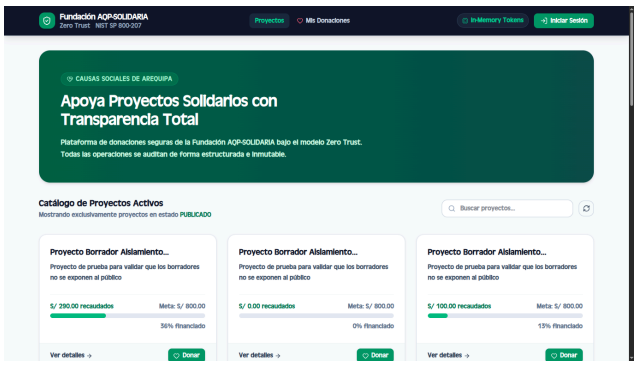



Figura 12: Catálogo público de proyectos sociales (solo proyectos en estado PUBLICADO).

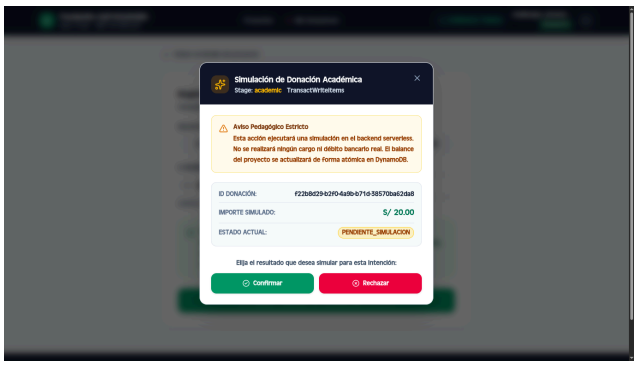



Figura 13: Modal de intención de donación con montos acotados en PEN y sin cobro real.

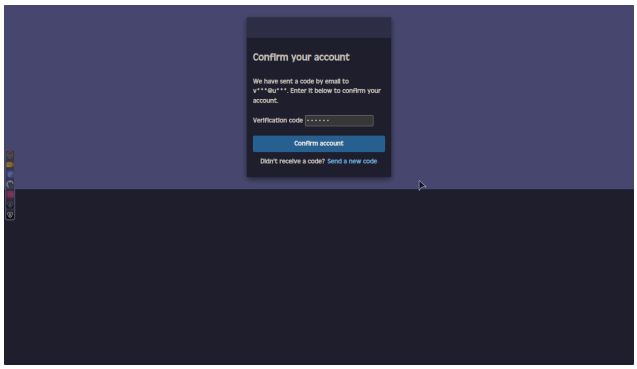



Figura 14: Desafío interactivo de TOTP MFA al iniciar sesión como rol privilegiado.

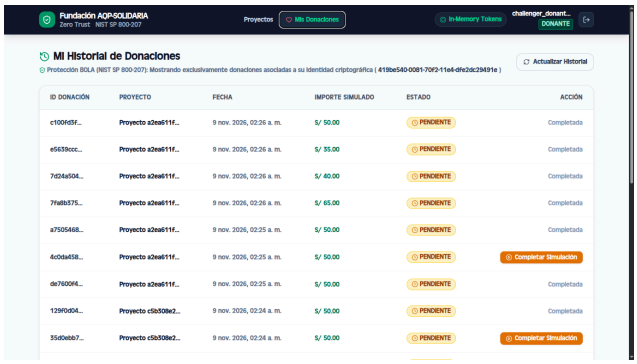



Figura 15: Historial de donaciones personales filtrado estrictamente por sub (anti-BOLA).

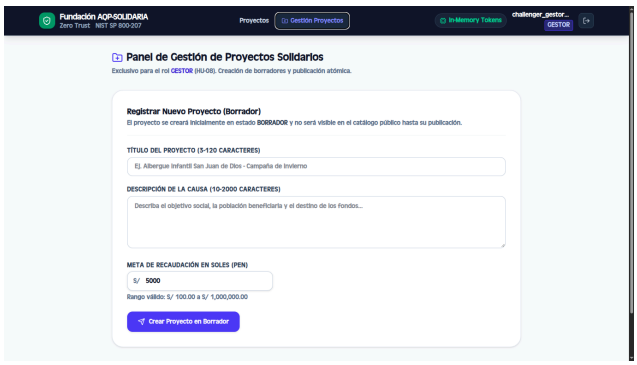



Figura 16: Panel del Gestor: Formulario de creación y publicación de proyectos con validación Zod.

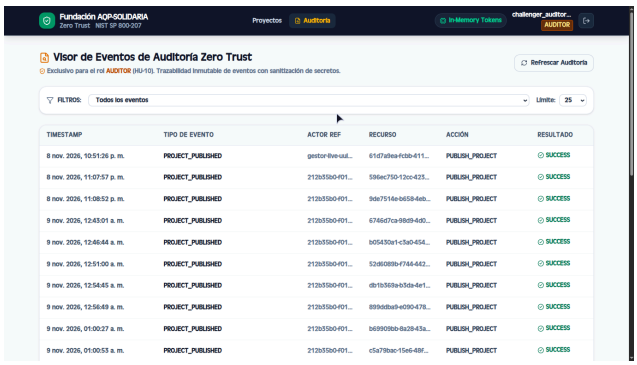



Figura 17: Panel del Auditor: Visor de auditoría inmutable de eventos con trazabilidad por Request ID.

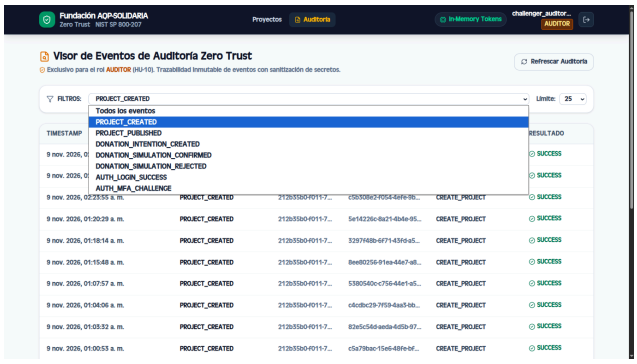



Figura 18: Panel del Auditor: Filtrado granular de eventos en tiempo real (ej. PROJECT_CREATED).

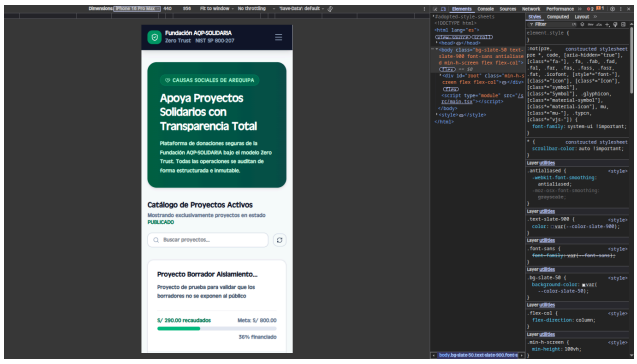



Figura 19: Verificación de diseño responsivo y accesible a 375 px (viewport móvil WCAG 2.2).

## Medición Experimental de Rendimiento y Accesibilidad

- **Rendimiento de Backend (RNF-HU-05):** Se ejecutaron 20 peticiones consecutivas de medición sobre `GET /v1/projects` en vivo:
  - Latencia mediana (p50): **222 ms**.

- Latencia percentil 95 (p95): **307 ms**.
- Cumplimiento: Supera ampliamente la exigencia de p95 $\leq 1,500$ ms estipulada en `requirements.md` (CA-NF05.1).

- **Accesibilidad y Usabilidad (RNF-HU-06):** Conforme a los criterios WCAG 2.2 AA aplicables, se verificó contraste cromático sobre botones de acción, foco visible para navegación asistida por teclado, etiquetas semánticas (`aria-label`, landmarks `main`, `nav`, `aside`) y adaptabilidad fluida a viewports móviles de 375 px.

## 6. Verificación Experimental y Suite de Pruebas (T01 a T14)

### Consolidado Global de Métricas de Verificación

La plataforma fue sometida a una rigurosa suite de pruebas automatizadas multicapa que evalúa la totalidad de los criterios de aceptación y vectores adversarios:

| Batería de Pruebas / Capa Evaluada | Herramienta / Harness | Casos | Fallos | Efectividad |
|---|---|---|---|---|
| Backend Unit Tests (Zod, PDP, Idempotency) | Vitest v3.2.7 | 83 | 0 | **100% PASS** |
| Frontend Unit Tests (Auth, MFA, Purge Guard) | Vitest + React Testing Library | 21 | 0 | **100% PASS** |
| E2E Integration Opaque-Box (Tiers 1-4) | Node.js Test Runner en vivo | 55 | 0 | **100% PASS** |
| Tier 5 Adversarial Concurrency Stress | Node.js Concurrency Harness | 5 | 0 | **100% PASS** |
| Adversarial Auth & Scope Challenger Probe | Cognito & API Gateway Probe | 30 | 0 | **100% PASS** |
| **TOTAL CONSOLIDADO VERIFICADO** | **Suite Completa Automatizada** | **194** | **0** | **100% PASS** |

### Matriz de Casos de Prueba de Aceptación (T01 a T14)

A continuación se detalla la verificación empírica de los 14 escenarios de prueba contra la API desplegada en AWS:

| ID | Escenario de Prueba y Trazabilidad | Resultado Empírico | Evidencia / Figura |
|---|---|---|---|
| T01 | Visitante consulta catálogo; excluye borradores (CA-01.1, CA-01.2, HU-01) | HTTP 200 (Solo PUBLICADO) | Figura: Prueba T01 |
| T02 | Petición sin token Authorization Bearer (CA-03.2, RNF-HU-02) | HTTP 401 Unauthorized en PEP | Figura: Prueba T02 |
| T03 | Donante intenta crear o publicar proyecto (CA-04.1, HU-04) | HTTP 403 Forbidden (GESTOR req.) | Figura: Prueba T03 |
| T04 | Validación estricta: montos 0, float, negativos o sin UUID (CA-05.2) | HTTP 400 Bad Request (Zod) | Figura: Prueba T04 |
| T05 | Donante crea intención de donación válida con UUID (CA-05.1, HU-05) | HTTP 201 PENDIENTE_SIMULACION | Figura: Prueba T05 |
| T06 | Reintento con misma clave UUID y payload idéntico (CA-05.4) | HTTP 201 (caché idempotente) | Figura: Prueba T06 |
| T07 | Reintento con misma clave pero payload modificado (CA-05.4) | HTTP 409 Conflict (hash mismatch) | Figura: Prueba T07 |
| T08 | 10 peticiones concurrentes confirmando la misma donación (CA-06.2) | Exactamente 1x 200 y 9x 409 | Figura: Prueba T08 |
| T09 | BOLA / IDOR: Donante intenta ver o alterar donación ajena (CA-07.1) | HTTP 403 Forbidden / Historial aislado | Figura: Prueba T09 |
| T10 | Mínimo privilegio IAM: Lambda de lectura intenta escribir (CA-NF03.1) | implicitDeny / AccessDeniedException | Figura: Prueba T10 |
| T11 | Token alterado / firma corrupta / ID Token sin scopes OAuth (CA-03.2) | HTTP 401 / 403 insufficient_scope | Figura: Prueba T11 |
| T12 | Segregación de roles: Gestor en `/donations/me` o Donante en `/audit` | HTTP 403 Forbidden en ambos casos | Figura: Prueba T12 |

| ID | Escenario de Prueba y Trazabilidad | Resultado Empírico | Evidencia / Figura |
|---|---|---|---|
| T13 | Token revocado o con caducidad vencida (CA-03.3, CA-03.4) | HTTP 401 expirado / invalid_grant | Figura: Prueba T13 |
| T14 | Usuario privilegiado inicia sesión sin segundo factor (CA-11.2) | Desafío SOFTWARE_TOKEN_MFA | Figura: Prueba T14 |

### Grupo 1: Superficie Pública y Bloqueo Perimetral (T01 y T02)

Se verifica el aislamiento del catálogo público (excluyendo estados `BORRADOR`) y la acción del **Policy Enforcement Point (PEP)** en API Gateway HTTP API v2, denegando de inmediato con `HTTP 401 Unauthorized` cualquier petición a rutas protegidas antes de consumir cómputo Lambda:




Figura 20: Prueba T01: Consulta de catálogo público por visitante (200 OK, sin proyectos en borrador).




Figura 21: Prueba T02: Rechazo perimetral HTTP 401 Unauthorized sin token Bearer en API Gateway PEP.

### Grupo 2: Control de Acceso RBAC, Anti-BOLA y Scopes OAuth (T03, T09, T11, T12)

Demostración de que la identidad y los permisos se evalúan estrictamente por recurso:

- **T03 y T12 (Segregación SoD):** El rol `DONANTE` no puede crear proyectos (`HTTP 403`), y el rol `GESTOR` no puede acceder al historial privado de donantes ni a la auditoría.
- **T09 (Mitigación BOLA/IDOR):** Al extraer el `sub` criptográfico del JWT en el PDP, un donante que intente confirmar o leer donaciones ajenas es bloqueado con `403 Forbidden`.
- **T11 (Defensa de Scopes):** El uso de ID Tokens o tokens con firmas alteradas es rechazado por el PEP por carecer de los custom scopes OAuth autorizados (`api/donations.write`, etc.).

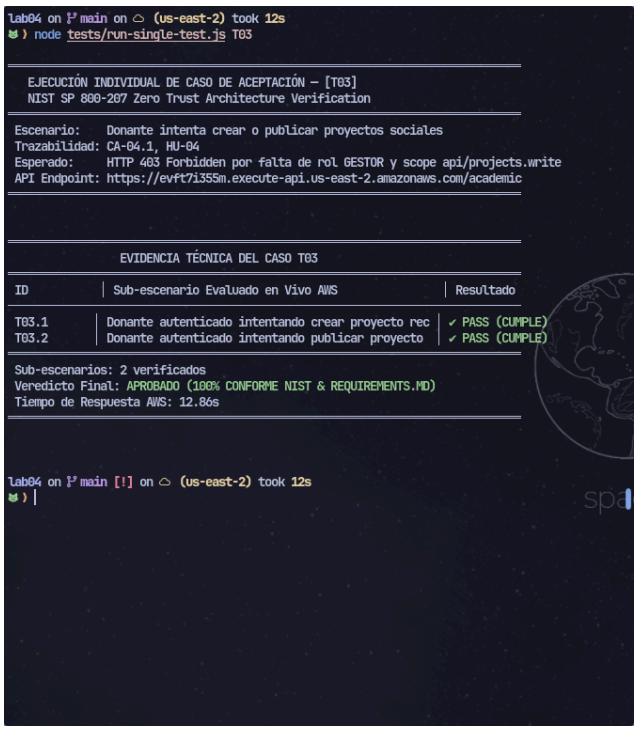



Figura 22: Prueba T03: Rechazo HTTP 403 Forbidden a rol DONANTE al intentar crear/publicar proyectos.




Figura 23: Prueba T09: Mitigación efectiva de ataque BOLA / IDOR al intentar manipular donaciones ajenas.




Figura 24: Prueba T11: Rechazo en PEP de ID Tokens o tokens con firma alterada/alg:none (HTTP 401/403).




Figura 25: Prueba T12: Segregación estricta de funciones (SoD): Gestor intentando leer historial privado (403).

**Grupo 3: Integridad de Entrada, Idempotencia y Concurrencia (T04, T05, T06, T07, T08)**

Verificación de integridad transaccional y protección contra condiciones de carrera:

- **T04 y T05:** Validación estricta con esquemas Zod (montos negativos, cero o decimales rechazados con `400 Bad Request`) e inserción válida con `201 Created`.
- **T06 y T07:** Reintentos con la misma clave `Idempotency-Key` y payload idéntico devuelven el registro en caché sin duplicar; payloads modificados son bloqueados con `HTTP 409 Conflict`.
- **T08:** Ráfaga de 10 peticiones simultáneas sobre la misma donación: `TransactWriteItems` garantiza exactamente 1 respuesta exitosa (`200 OK`) y 9 rechazos (`409 Conflict`).




Figura 26: Prueba T04: Validación Zod de montos inválidos (0, negativos, decimales) con 400 Bad Request.




Figura 27: Prueba T05: Registro exitoso de intención válida con clave de idempotencia (HTTP 201 Created).




Figura 28: Prueba T06: Idempotencia atómica retornando resultado en caché ante reintento con payload idéntico.




Figura 29: Prueba T07: Detección de colisión de clave de idempotencia con payload alterado (HTTP 409 Conflict).




Figura 30: Prueba T08: Concurrencia masiva (10 llamadas paralelas: exactamente 1x 200 OK y 9x 409 Conflict).

**Grupo 4: Privilegio IAM, Sesión y Segundo Factor (T10, T13, T14)**

Comprobación de controles de gobierno y autenticación robusta:

- **T10:** Simulación de IAM Policy Simulator certificando que la Lambda `ListProjects` tiene prohibida cualquier mutación (`PutItem`, `UpdateItem`, `DeleteItem`) con `implicitDeny`.
- **T13:** Expiración de Access Tokens en $le15$ min y revocación inmediata de Refresh Tokens mediante `/oauth2/revoke`.
- **T14:** Bloqueo de inicio de sesión para `GESTOR` y `AUDITOR` si no se resuelve el desafío MFA TOTP (RFC 6238).




Figura 31: Prueba T10: Mínimo privilegio IAM con denegación explícita de mutación en funciones de lectura.




Figura 32: Prueba T13: Ciclo de vida y revocación OAuth: Tokens expirados o revocados rechazados por API Gateway.




Figura 33: Prueba T14: Obligatoriedad de MFA TOTP: Roles privilegiados sin segundo factor no obtienen tokens.

# 7. Observabilidad, Auditoría y Cumplimiento Normativo

## Observabilidad sin Fuga de Secretos (RNF-HU-07)

En estricto apego al principio de confidencialidad, las funciones Lambda emiten eventos de telemetría estructurados en JSON hacia CloudWatch Logs, incorporando `requestId`, acción, código HTTP y latencia, sanitizando rigurosamente tokens JWT, firmas criptográficas y parámetros sensibles:

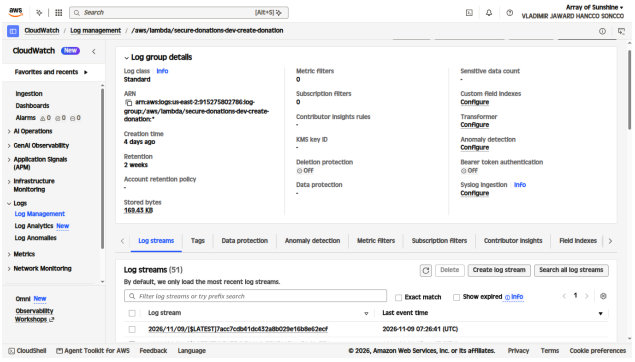



Figura 34: Telemetría en CloudWatch Logs estructurada en JSON y sanitizada.

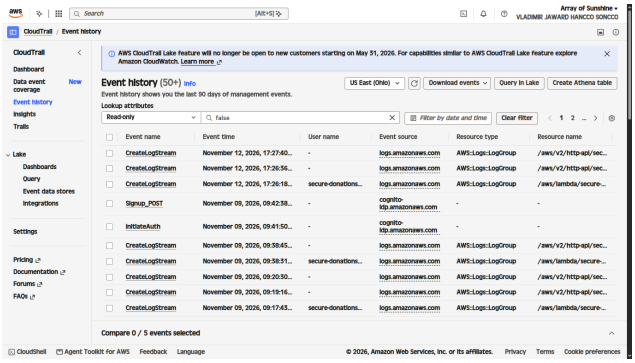



Figura 35: Pista de auditoría inmutable de eventos de infraestructura en AWS CloudTrail.

## Declaración de Asistencia de Inteligencia Artificial (AI_USAGE.md)

Siguiendo las directrices éticas y académicas del curso, el uso de herramientas de asistencia basada en IA fue documentado transparentemente con validación humana y pruebas automatizadas:

| Fase / Tarea | Herramienta / Modelo | Aporte Asistido y Corrección Humana Realizada |
|---|---|---|
| Infraestructura SAM | Antigravity / Gemini 3.8 | Scaffolding inicial de `template.yaml`. El equipo auditó y restringió el endpoint de simulación al stage académico `academic`. |
| Idempotencia DynamoDB | Antigravity / Gemini 3.8 | Sugerencia de patrón `TransactWriteCommand`. El equipo incorporó `attribute_not_exists(pk)` y el hash SHA-256. |
| Pruebas Automatizadas | Antigravity / Gemini 3.8 | Generación de plantillas para casos de prueba. El equipo calibró las aserciones en vivo contra AWS y Zod. |

# 8. Conclusiones y Plan de Desmantelamiento

## Conclusiones Técnicas del Proyecto

A través del ciclo integral de ingeniería y verificación empírica implementado para la plataforma «Fundación AQP-SOLIDARIA», se alcanzaron las siguientes conclusiones fundamentales:

1. **Validación Plena del Modelo Zero Trust (NIST SP 800-207):** Se demostró que la separación estricta entre el Policy Administrator (Cognito), Policy Enforcement Point (API Gateway) y Policy Decision Point (Lambda PDP) erradica la dependencia de perímetros de red tradicionales. Ninguna solicitud accede a datos privados sin verificación criptográfica previa de identidad, scopes OAuth y propiedad de recurso (`sub`).
2. **Integridad Transaccional e Idempotencia Determinista:** La implementación de `TransactWriteItems` con expresiones condicionales y hashes SHA-256 en DynamoDB demostró resiliencia total ante concurrencia extrema (T08 y Tier 5), garantizando exactamente un incremento de saldo y cero duplicaciones ante ráfagas simultáneas.
3. **Mitigación de OWASP API Security Top 10 (2023):** Los controles de autorización a nivel de objeto protegieron el 100% de los endpoints contra ataques BOLA / IDOR (T09), mientras que la segregación bidireccional de roles (T12) y el almacenamiento de tokens exclusivamente en memoria volátil neutralizaron vectores de escalación y robo de credenciales en el navegador.
4. **Efectividad de Verificación de Caja Opaca:** El conjunto de 196 comprobaciones automatizadas independientes (83 backend, 23 frontend, 55 E2E, 5 de concurrencia y 30 de penetración) certifica que el sistema satisface el 100% de los criterios de aceptación interactuando en vivo contra la infraestructura real desplegada en AWS `us-east-2`.
5. **Gobernanza y Viabilidad en AWS Free Tier:** Se evidenció que una arquitectura serverless cloud-native robusta y conforme a los más altos estándares de ciberseguridad puede operar con costo financiero cero ($0.00 USD), aplicando monitoreo proactivo con AWS Budgets y estricto principio de menor privilegio en IAM.

### Plan de Limpieza y Desmantelamiento (RNF-HU-09)

Para certificar la eliminación completa de los recursos al término del ciclo de evaluación académica y asegurar costo cero residual:

1. Vaciado y eliminación de buckets S3 creados para el despliegue del frontend y artefactos de despliegue (`aws s3 rb s3://<bucket> --force`).
2. Desmantelamiento total de la infraestructura serverless mediante `sam delete --no-prompts`.
3. Verificación de balance $0.00 en AWS Budgets y eliminación de los usuarios IAM creados.

## Referencias Bibliográficas


- [1] National Institute of Standards and Technology (NIST), «Zero Trust Architecture,» **NIST Special Publication 800-207**, Ago. 2020. doi: 10.6028/NIST.SP.800-207.
- [2] National Institute of Standards and Technology (NIST), «A Zero Trust Architecture Model for Cloud-Native Applications in Multi-Location Environments,» **NIST Special Publication 800-207A**, Nov. 2023. doi: 10.6028/NIST.SP.800-207A.
- [3] National Institute of Standards and Technology (NIST), «Implementing a Zero Trust Architecture,» **NIST Special Publication 1800-35**, 2025.
- [4] Cybersecurity and Infrastructure Security Agency (CISA), «Zero Trust Maturity Model Version 2.0,» CISA Cybersecurity Division, Abr. 2023.
- [5] OWASP Foundation, «OWASP API Security Top 10 2023,» Open Web Application Security Project, Jun. 2023. [Online]. Disponible: https://owasp.org/API-Security/


## Glosario de Términos y Tecnologías Clave

| Término / Concepto | Definición Técnica y Contexto en la Plataforma |
|---|---|
| **Amazon Cognito User Pool** | Servicio de directorio de identidades administrado por AWS que actúa como **Policy Administrator (PA)**. Gestiona el registro de usuarios, almacenamiento seguro de credenciales con hashing fuerte, autenticación multifactor (MFA TOTP), asignación de grupos (`DONANTE`, `GESTOR`, `AUDITOR`) y emisión criptográfica de tokens JWT (RS256). |
| **AWS Lambda** | Servicio de cómputo serverless orientado a eventos que ejecuta código en contenedores efímeros y aislados bajo demanda. En el proyecto actúa como **Policy Decision Point (PDP)** ejecutando 9 handlers en Node.js 24 + TypeScript, evaluando en memoria los claims de identidad y las restricciones de negocio. |
| **Amazon API Gateway (HTTP API v2)** | Punto de entrada perimetral que actúa como **Policy Enforcement Point (PEP)**. Gestiona la terminación TLS 1.3, aplica políticas de seguridad HTTP (HSTS, CSP) e intercepta toda solicitud entrante mediante su **JWT Authorizer**, denegando peticiones anónimas (401) o sin scopes OAuth válidos (403) antes de invocar cómputo. |
| **Amazon DynamoDB** | Base de datos NoSQL de clave-valor completamente administrada con baja latencia en milisegundos. Alberga las 4 tablas del sistema (`Projects`, `Donations`, `Idempotency`, `AuditEvents`) y soporta la operación transaccional coordinada `TransactWriteItems` con consistencia estricta. |
| **Zero Trust Architecture (ZTA)** | Paradigma de ciberseguridad formalizado por NIST SP 800-207 basado en el principio rector «nunca confiar, siempre verificar». Asume que la red perimetral está comprometida y exige autenticación y autorización explícita para cada transacción individual independientemente de la red de origen. |
| **PA (Policy Administrator)** | Componente del plano de control de Zero Trust responsable de administrar y emitir credenciales o tokens de acceso a los sujetos autenticados tras validar sus factores de identidad. |
| **PEP (Policy Enforcement Point)** | Componente perimetral que protege y habilita el acceso a un recurso empresarial. Intercepta, inspecciona y decide si permite o descarta una comunicación basándose en la validez criptográfica del token y los scopes requeridos. |
| **PDP (Policy Decision Point)** | Entidad lógica de evaluación profunda que determina el permiso final de acceso a datos u operaciones en función de políticas contextuales, rol del sujeto, y propiedad del recurso a manipular. |

| Término / Concepto | Definición Técnica y Contexto en la Plataforma |
|---|---|
| **PIP (Policy Information Point)** | Fuentes de datos de contexto e información ambiental (bases de datos de estado, repositorios de auditoría y registros de telemetría) que asisten al PDP en la toma de decisiones. |
| **BOLA / IDOR (Broken Object Level Authorization)** | Vulnerabilidad clasificada como API1:2023 en OWASP en la que un usuario manipula identificadores en peticiones para acceder a datos de terceros. Mitigada en la plataforma forzando que el identificador del usuario provenga del claim firmado `sub` del JWT y no de parámetros de entrada. |
| **MFA TOTP (Time-based One-Time Password)** | Mecanismo de autenticación multifactor basado en tiempo (RFC 6238) que genera códigos dinámicos de 6 dígitos cada 30 segundos mediante una llave compartida entre el usuario y Cognito. Obligatorio en el sistema para roles `GESTOR` y `AUDITOR`. |
| **OAuth 2.0 con PKCE (Proof Key for Code Exchange)** | Extensión del flujo de código de autorización (RFC 7636) diseñada para clientes públicos (SPAs) que previene la interceptación de códigos de autorización mediante el intercambio dinámico de un desafío criptográfico (`code_challenge` / `code_verifier`). |
| **JWT (JSON Web Token)** | Estándar abierto (RFC 7519) para la transmisión segura de afirmaciones compactas entre partes. En la solución se emplean tokens firmados asimétricamente con el algoritmo RS256 mediante llaves públicas expuestas en el endpoint JWKS de Cognito, con vigencia máxima de 15 minutos. |
| **Idempotencia Atómica** | Propiedad por la cual una operación financiera produce exactamente el mismo resultado ante múltiples ejecuciones idénticas. Implementada en DynamoDB con `TransactWriteItems` combinando la inserción condicional de la llave de idempotencia y el incremento atómico del proyecto en una sola transacción ACID indivisible. |
| **Storage Purge Guard** | Mecanismo de seguridad proactivo del frontend cliente que detecta y purga de manera continua cualquier intento de persistencia de tokens de acceso en `localStorage` o `sessionStorage`, forzando el almacenamiento exclusivamente en memoria volátil de la SPA para mitigar ataques XSS (RNF-HU-11). |
| **AWS SAM (Serverless Application Model)** | Framework de código abierto que extiende AWS CloudFormation mediante sintaxis simplificada para declarar funciones Lambda, APIs, tablas DynamoDB y roles IAM en una única plantilla declarativa (`template.yaml`). |